Khayyam Math: a voice-narrated math tutor with multi-tool figure routing and vision-audited generation

Arash Kermani Kolankeh

Corresponding author(s). E-mail(s): arash_kermani@yahoo.com;

Abstract

Producing a tutor-quality mathematical figure from a learner's natural-language prompt and synchronising it with a spoken walkthrough is harder than producing either alone: clean labelled figures routinely ship with narrations that contradict their own pixels, and a single end-to-end large-language-model (LLM) call rediscovers, badly, layout problems that mature symbolic tools have solved for decades. We describe a deployed tutoring system that turns a single natural-language prompt into a labelled Scalable Vector Graphics (SVG) figure synchronised with a phrase-timed spoken walkthrough the learner can interrupt at any moment. A multi-tool routing layer dispatches each prompt to deterministic Python templates, the Graphviz layout engine, a matplotlib backend, or a vision-audited LLM-SVG path with structured-fix retries. A multi-tier symbolic verifier (SymPy through SMT solving to a Lean 4 kernel) translates every claim the model emits into a proof obligation and blocks the figure from being shown on any disproved claim. A browser-side viewer binds the spoken script to stable element identifiers, plays multi-element highlights at phrase boundaries, and pauses mid-phrase on learner activity. On a diverse benchmark, the GPT-4o express backend wins under blinded multimodal judging, and Graphviz routing reduces graph-shaped end-to-end latency by an order of magnitude. The symbolic verifier resolves a large majority of emitted claims at the cheapest tier and has converted dozens of systemic LLM failure modes into named regression checks. We also report a negative result on neural layout correction: a graph-conditioned graph-neural-network delta-predictor and a discrete-diffusion denoiser both fail to outperform a no-op baseline, while a small learned re-ranker on top of a constraint solver does help. The system has run as a public service since 2026-05-10 at https://khayyammath.com under MIT licence (source at https://github.com/khayyam-math/khayyam-math); an offline-capable Qwen2.5-7B LoRA adapter is published as a side artefact on the project's Hugging Face organisation. A classroom evaluation with learners is planned future work.

Keywords: intelligent tutoring systems, large language models, multimodal learning, mathematical visualisation, narrated worked examples, symbolic verification, voice interaction

1 Introduction

A first-year computer-science student stuck on the question *"how does 3SAT reduce to vertex cover, with a worked example?"* is asking for two things at the same time. The student needs a clean, labelled picture of the reduction — the original Boolean clauses, the literal vertices the construction creates, the satisfying assignment turned into a vertex cover — and the student needs a teacher's voice next to that picture, walking through which element of the figure corresponds to which clause, in the order the construction builds them up. A chat-only large language model (LLM) can write a passable English explanation but cannot show the figure. A diagram tool can show a figure but cannot speak. An end-to-end multimodal model can do both at once but hallucinates labels, draws geometric primitives that wander off the canvas, lays its captions on top of the very shapes they are meant to label, and routinely narrates a claim its own figure does not actually depict. A learner sitting in front of any one of these is a learner who is not getting tutored.

The system we describe in this paper was built to close that gap for an individual learner. A learner types or speaks one natural-language prompt, and within seconds the system shows a labelled mathematical figure on a live canvas while a synthesised teacher's voice walks through the figure. Every spoken phrase highlights the elements of the figure it is currently discussing; if the learner wants to ask a follow-up, focusing the chat box pauses the narration mid-phrase so the figure is not still being explained while the learner is trying to type. Behind the scenes, every numerical claim the system is about to speak is run through a symbolic verifier — SymPy first, then an SMT solver, then the Lean 4 kernel — and the figure is blocked from reaching the learner if any tier disproves a claim. The mathematics-correctness layer is what lets the system be trusted; the synchronised narration is what makes the figure feel like a worked example rather than a static diagram; the multi-tool router is what makes both fast enough to leave running in front of a real learner.

The system, named *Khayyam Math*, has been live as a public web service since 2026-05-10 at https://khayyammath.com. The complete source code, the AWS CDK infrastructure stack, training corpora and per-prompt evaluation artefacts are released under the MIT licence at https://github.com/khayyam-math/khayyam-math; trained LoRA adapters are published at https://huggingface.co/khayyam-math. A companion preprint of the deterministic predecessor system, *SeVim*, is deposited on Zenodo under DOI 10.5281/zenodo.20367147.

What the learner experiences.

From the learner's seat, the system is a single web page: a chat surface on the left, a live diagram canvas on the right. Typing *"show me the unit circle with* sin *and* cos *labelled at* 30°*,* 45°*,* 60°*"* starts a render within seconds; the canvas fades in a unit

circle with three radii at the requested angles, each annotated with its sine and cosine value computed in Python rather than guessed by an LLM, while the synthesised voice walks through which value belongs to which radius. Asking *"now add the median from A"* after a triangle figure does not replay the prior lesson: the figure stays exactly as it was and gains the median, and the narration speaks only the new content. Activating the microphone, focusing the textbox, or pressing send pauses the audio so the learner is not lectured over while asking the next question.

Why this is not just an LLM call.

Three failure modes of end-to-end LLM figure generation force the architecture we describe. First, frontier LLMs continue to confidently emit mathematically wrong claims at non-trivial rates (Guan et al. 2024), so a tutoring system cannot rely on the model's own self-assessment; we drive every claim through a chain of decidable symbolic checks. Second, an LLM asked to lay out a six-state deterministic finite automaton without crossings, or to fit a regression scatter inside its axes, recovers badly a layout problem that Graphviz and matplotlib have owned for decades; we route the right prompts to the right engine and ask the LLM only for the residual nobody has solved. Third, the act of *binding* a narration phrase to the shape it is currently describing must be deterministic — a mistimed highlight is worse than no highlight at all — and the model is unreliable about emitting the stable element identifiers the viewer needs, so a deterministic pass injects identifiers and grounds each phrase to its element after the LLM returns.

What this paper contributes.

The paper's contributions are organised around three axes: what the learner sees, what gets verified before it reaches the learner, and what we have learned about where neural and symbolic components should sit on top of each other.

- **Multi-tool routing for figure generation** (Section 5). Each prompt is dispatched to one of four execution paths: deterministic Python templates for the matrix, geometry, and elementary-arithmetic families (Section 5.3); the Graphviz layout engine for graph-shaped figures including deterministic finite automata, Turing machines, directed acyclic graphs, trees, and category-theoretic commutative diagrams (Section 5.1); a matplotlib backend for plot-shaped figures and three-dimensional surfaces (Section 5.2); or a vision-audited LLM-SVG path with structured-fix retries (Section 4.1) for everything else. The router is the largest single quality lever in the system because the LLM is asked to draw only the residual that mature symbolic tools cannot.
- **Symbolic math-correctness verification on the synchronous request path** (Section 4.5). A five-tier chain (SymPy through Z3 SMT through the Lean 4 kernel through per-domain structural checkers through an offline Mathlib catalog, in cost order) translates every claim the model emits into the cheapest decidable obligation that can resolve it. A tier that *disproves* a claim blocks the figure at the server; a tier that fails to decide defers to the next. This is the only mechanism that catches a clean figure whose narration is mathematically false — a failure mode that no aesthetic vision audit can detect.

- **Browser-side narration binding and learner-interrupt** (Sections 4.2 and 4.3). The viewer streams figure updates over Server-Sent Events, plays a phrase-timed walkthrough whose boundaries are accurate to the WAV header rather than estimated from word counts, applies multi-element highlights at those boundaries, and pauses mid-phrase the moment the learner focuses the chat box, types a character, activates the microphone, or hits `Send`. A deterministic narration-binding pass injects stable element identifiers when the LLM omits them and grounds each spoken phrase to a real figure element, so the highlight cursor cannot drift away from what the audio is discussing.
- **Figure-quality hardening driven by an adversarial stress test** (Section 7). A browser-grade vision auditor (headless Chromium rather than `cairosvg`, so the critic sees what the learner sees), a family of idempotent layout-repair post-processors, and explicit prompt discipline against three named LLM failure modes (visual padding, concept substitution, premature finish) raise the figure-quality floor on the LLM-SVG path.
- **An open negative result on neural layout correction** (Section 8). A graph-conditioned graph-neural-network delta-predictor and a LayoutDM-style discrete-diffusion denoiser, both trained on tens of thousands of (broken, fixed) layout pairs harvested from the production loop, fail to outperform the trivial no-op baseline. The one neural component that does land a measurable win is a small graph-conditioned binary quality scorer used as a re-ranker over a CP-SAT label-placement planner — a focused application of learning on top of a mature symbolic baseline.
- **Closed-loop distillation as a side artefact** (Section 9). Every accepted turn is captured to telemetry and periodically becomes a training example for a LoRA adapter on `Qwen2.5-7B-Instruct` (Hu et al. 2022; Yang et al. 2024), published on Hugging Face for offline and air-gapped deployment. The live production service uses GPT-4o; the open-model adapter is a side artefact, not a replacement.
- **A reproducible empirical study** (Section 10) with a blinded multimodal judge on a diverse prompt set, a Lean-graded verifier sweep at a four-figure scale, and a held-out vision-audit benchmark across the four routes plus the scorer re-ranker, framed around learner-perceivable outcomes (figure quality, math correctness, latency under student attention) rather than internal system metrics alone.

The paper concentrates on what the system was built to deliver to an individual learner, and is honest about what it has not yet measured: every claim about figure quality, mathematical correctness, and latency in this paper comes from automated benchmarks and from production telemetry, not from a controlled study with student users. A classroom pilot is planned future work and is described as such in Section 10.7.

2 Background and Related Work

Intelligent tutoring systems and the figure problem.

The field of intelligent tutoring systems (ITS) traces a forty-year arc from rule-based geometry and algebra tutors (Anderson et al. 1985) through deployed cognitive tutors that reach millions of school students (Koedinger et al. 1997), dialogue-based natural-language tutors such as AutoTutor (Graesser et al. 2004), and the meta-analyses

comparing human, computer-based, and group instruction (VanLehn 2011, 2006); the long-standing reference point for the question of how close machine tutoring can come to a one-on-one human tutor is the so-called two-sigma effect described by Bloom (1984). Throughout this arc, the systems excel where the domain has well-defined problem schemata (algebra, equation solving, propositional logic) and the tutor can compose corrective feedback from rules. Where they have historically been weaker is in the act of *showing* the learner a custom mathematical figure on demand: pre-authored figures are dependable but expensive to produce and cannot cover an open-ended prompt; rule-based diagram generators handle a narrow domain at the cost of months of authoring per domain.

A second strand of work targets exactly this gap by replacing or augmenting handcrafted feedback with LLM-driven generation: review articles enumerate the opportunities and risks of layering LLMs onto educational scaffolding (Kasneci et al. 2023; García-Méndez et al. 2024), while benchmarking work suggests that current LLMs match the *appearance* of tutoring better than they match its *adaptivity* (Borchers and Shou 2025). Public products in this space include Khan Academy's Khanmigo (Khan Academy 2024) and the older Wolfram—Alpha computational-knowledge engine (Wolfram 2009), both of which serve a learner-facing audience at scale but treat figure generation either as a manual asset or as a fixed-template plot from a structured query. Our system sits one step further along the same arc: a single natural-language prompt yields a custom labelled SVG figure plus a synchronised narration, with mathematical correctness gated by a symbolic verifier rather than left to the model.

Worked examples, cognitive load, and multimedia learning.

The pedagogical justification for phrase-aligned narration over a labelled figure is older than the technology that delivers it. The worked-examples literature establishes that a step-by-step demonstration with the steps made explicit reduces extraneous cognitive load and improves transfer relative to unguided problem solving (Sweller 1988; Renkl 2014), and Mayer's multimedia-learning principles (Mayer 2009) prescribe (among other things) the *spatial and temporal contiguity* of words and pictures: a spoken explanation that names the same element the learner is currently looking at outperforms the same explanation delivered out of sync. The phrase-timed highlight cursor described in Section 4.2 is a direct application of contiguity to a LLM-generated figure.

Vector-graphics generation with neural models.

A first wave of work optimises differentiable rasterisations of SVG against a 2D image objective: VectorFusion (Jain et al. 2023) and SVGDreamer (Xing et al. 2024) guide a Bézier-curve representation through a frozen text-to-image diffusion model. A second wave directly emits SVG *tokens*: IconShop (Wu et al. 2023), StarVector (Rodriguez et al. 2025a), and OmniSVG (Yang et al. 2025) train autoregressive transformers to predict SVG tag sequences. A third line uses LLMs as planners or programmers: DiagrammerGPT (Zala et al. 2024) has an LLM plan a diagram, render it through a downstream system, and refine; Chat2SVG (Wu et al. 2025) and LLM4SVG (Xing

et al. 2025) ask an LLM directly for SVG code. Reason-SVG (Xing et al. 2026) and rendering-aware reinforcement learning (Rodriguez et al. 2025b) add a renderer-in-the-loop reward. Our system sits in the third family: it asks a large multimodal model for SVG *plus* a narration in one structured response, but unlike the cited works it gives the model a render-to-PNG vision audit as a deterministic, structured-output critic rather than as a learned reward.

Diagram synthesis from formal notation.

Penrose (Ye et al. 2020) compiles abstract mathematical statements into diagrams via constrained numerical optimisation; Graphviz (Ellson et al. 2004) renders concept graphs through layered or force-directed layout algorithms (Sugiyama et al. 1981). Our system retains a Sugiyama-style layout option for non-positional concept diagrams, but the figure types we target in this paper (matrix multiplication grids, reductions between problems in computational complexity, Bayes-theorem layouts) are awkward to express as constraint problems and are precisely where LLM-driven generation has an opening.

LLMs as judges.

A separate line of work uses LLMs as judges for structured-output evaluation (Zheng et al. 2023; Guan et al. 2024); we follow this paradigm with a blinded multimodal judge in Section 10.4, while remaining aware that GPT-4o continues to hallucinate on a non-trivial fraction of inputs (Guan et al. 2024).

Tool-augmented LLMs.

The Model Context Protocol (Anthropic 2024) specifies a JSON-RPC contract between LLM clients (e.g. Claude Desktop, IDE plug-ins) and tool servers, generalising earlier tool-use work (Schick et al. 2023). An earlier system from the same authoring effort ships a Model Context Protocol server so that any compliant client can drive the diagram canvas directly; the system described in this paper bypasses the protocol overhead in favour of an in-process structured-output call when the user is interacting through our own chat surface.

Parameter-efficient fine-tuning.

LoRA (Hu et al. 2022) factors a frozen pre-trained weight matrix's update as the product of two low-rank matrices. We use the Hugging Face PEFT library (Mangrulkar et al. 2022) and TRL's SFTTrainer (von Werra et al. 2020) to adapt `Qwen2.5-7B-Instruct` (Yang et al. 2024) on the corpus that the system collects from itself.

3 Learning Design and Pedagogical Affordances

This section names the pedagogical commitments that drove the technical architecture described in the rest of the paper. These commitments are stated as design choices, not as empirical findings: the system instantiates them, but a controlled classroom evaluation of their effect on learning outcomes is left as future work (Section 10.7).

Phrase-aligned narration as a worked-example scaffold.

A worked example, in the cognitive-load sense (Sweller 1988; Renkl 2014), presents the learner with a fully solved problem and makes each solution step explicit. The figure-plus-narration pair this system delivers is a worked example in exactly that sense: the figure is the artefact of the solution, the narration enumerates the steps, and the highlight cursor implements Mayer's spatial-contiguity principle (Mayer 2009) by binding each spoken step to the figure element it currently concerns. The phrase boundaries are accurate to the WAV header (Section 4.2) rather than estimated from word counts, because cumulative drift between audio and visual cue degrades exactly the contiguity that the design relies on. A learner watching the unit-circle figure hears the sine value being named at the moment the corresponding radius lights up; the next phrase moves the highlight onto the cosine label as the audio names it. The system is not a slide-deck reader: each highlight is bound to its element by content-word overlap so that even when the LLM-emitted narration drifts in surface form, the cursor still resolves to the geometrically relevant element of the figure.

The mathematics-correctness verifier as misinformation control.

The figures a learner is most likely to study carefully are exactly the ones a tutoring system most needs to get right. A vision auditor that judges only aesthetic well-formedness cannot catch a clean triangle figure whose narration asserts that the interior angles sum to 2π rather than π — the figure passes the aesthetic rubric and the false claim is delivered confidently to the learner. The five-tier symbolic verifier of Section 4.5 sits on the synchronous request path precisely because tagging suspected hallucinations after the fact is not enough in deployment; a disproved claim must prevent the figure from reaching the learner at all. The asymmetry between *block on disprove* and *defer on undecided* is what allows the system to remain useful: a calculus prompt whose claim cannot be resolved by SymPy or Z3 is still shown to the learner, with the unverified claim tagged in the audit log for review, while a claim the verifier can refute is blocked. This is closer to the model of an editor with veto power over a teacher's slide deck than to the model of a teacher's own self-doubt.

Multi-route generation as adaptation to the mathematics being taught.

A tutoring system that produces a regression scatter by asking GPT-4o to draw SVG circles for points and a line element for the fit is, in practice, producing a worse figure than one that calls matplotlib with the same numerical content. A system that draws a deterministic finite automaton by asking an LLM to lay out states without crossings is producing a worse figure than one that emits DOT and lets Graphviz lay it out. The multi-tool router (Section 5) treats this as a pedagogical choice rather than as an engineering one: the right visual idiom for the mathematics in question is determined by classifying the prompt and dispatching it to the engine whose layout invariants match. The deterministic templates for the matrix family (Section 5.3) go a step further: every label that appears in a matrix-multiplication figure is computed in Python rather than guessed by an LLM, so the worked numerical example a learner sees in a 2×2 multiplication is correct by construction and cannot fall out of sync between figure and caption.

Learner-paced delivery through interrupt-on-activity.

The narration-interrupt described in Section 4.3 treats the audio as something the learner is allowed to override at any moment. The four interaction triggers (focusing the textbox, typing a character, activating the microphone, pressing `Send`) all pause the narration mid-phrase, so a learner who has formed a follow-up question is not still being lectured at while trying to type. This is the simplest form of learner pacing the architecture can support; explicit student-knowledge models and within-session adaptivity (VanLehn 2011; Borchers and Shou 2025) are out of scope for the current system. We return to this limit in Section 11.

What the system is not.

The system does not implement a student-knowledge tracer, does not maintain a per-learner skill model across sessions, and does not adjust the level of mathematical formality based on prior interaction. Each prompt is interpreted in isolation. We consider the figure-plus-narration response a substrate that a more conventional ITS could sit on top of, not a replacement for student modelling; bringing the two together is a natural direction for follow-on work that uses the same telemetry corpus as a per-session learning signal rather than a per-turn training signal.

4 System Architecture

Our system is structured as three in-process components and a browser frontend (Figure 1). A learner types or speaks into the chat surface; the chat backend resolves backend-only requests (highlight colour, audio speed) through a regex pre-router and otherwise forwards the prompt to the *express* path; the express path returns an SVG, a narration, and a title; the canvas service writes those to disk and broadcasts them via Server-Sent Events to the browser viewer; the viewer renders the SVG and plays the narration in lock-step with the highlight schedule.

4.1 The Express Path

The *express* path is the single tool that powers the chat surface. It receives the user prompt and any prior canvas identifiers the user has pinned, builds a strict JSON schema for the response (Listing 1), calls GPT-4o (OpenAI 2024a) with that schema enforced via OpenAI's structured-output mode (OpenAI 2024b), and then runs a vision-audit retry loop.

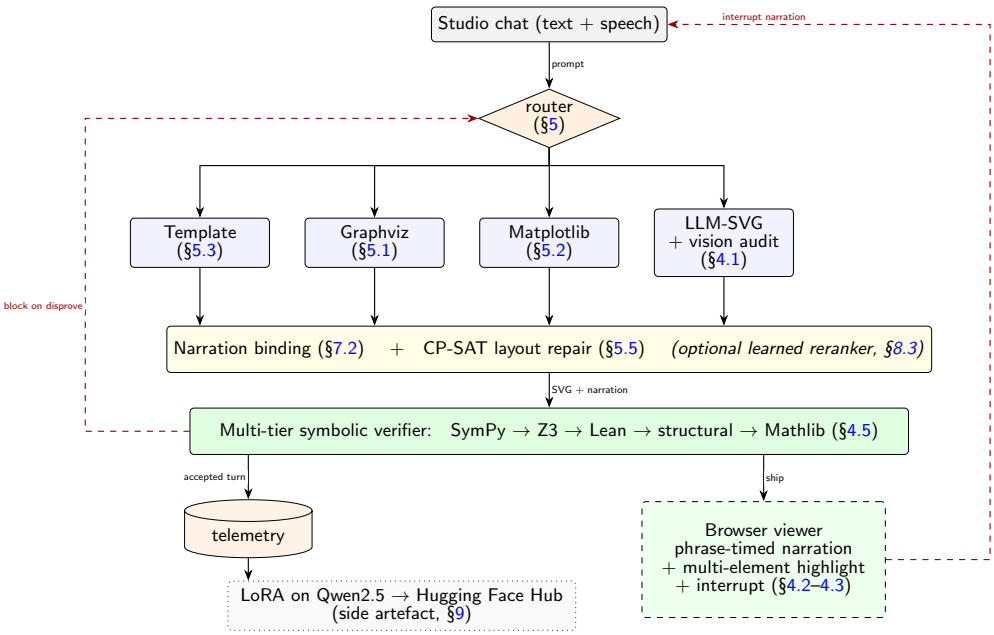


Fig. 1 End-to-end pipeline of the system, with every contribution from the introduction (§1) annotated with the section that describes it. A learner prompt enters the chat surface and is dispatched by the router (§5) to one of four execution paths — deterministic Python templates (§5.3), Graphviz (§5.1), matplotlib (§5.2), or a vision-audited LLM-SVG path with structured-fix retries (§4.1). All four paths converge on a layout-repair stage (narration binding §7.2 and CP-SAT label placement §5.5, optionally with the learned-scorer reranker of §8.3) and then a multi-tier symbolic verifier (§4.5) that blocks the figure from being shown if any tier disproves a narration claim. The browser viewer (§4.2–4.3) plays a phrase-timed walkthrough with multi-element highlights and accepts a chat-side interrupt. Every accepted turn is captured to telemetry and exported periodically to train the side-artefact LoRA adapter published on Hugging Face (§9); the live service does not depend on it. Dashed red arrows are feedback paths.

Listing 1 The structured-output schema enforced on every express call. The model cannot return text outside this contract; the OpenAI API rejects malformed responses before they reach the application.

```
{
  "svg":   "<svg xmlns='...' viewBox='0 0 W H' ...>...</svg>",
  "narration": [
    {"speak": "Here is the input clause C1...",
     "highlight": ["clause_c1", "literal_x1"]},
    ...
  ],
  "title": "3SAT to Vertex Cover"
}
```

Pedagogical system prompt.

The system prompt forces the model to (i) name the concept, (ii) include a worked numerical example with concrete values rather than only abstract symbols, (iii) walk

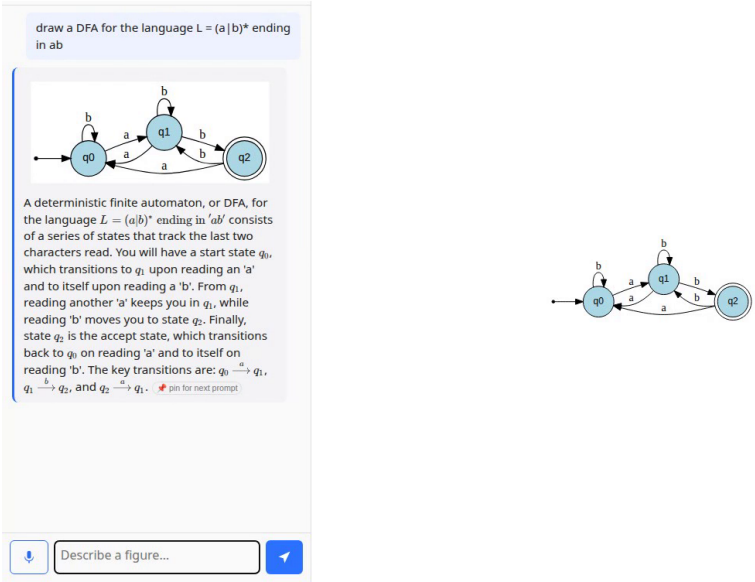


Fig. 2 The system in production, on the prompt *"draw a DFA for the language* $L = (a|b)^*$ *ending in ab"*. The left pane is the chat surface: it carries the user prompt, a thumbnail of the generated figure, and the spoken-aloud narration script. The right pane is the live SVG canvas, which renders the same figure at full size and is the surface against which the audio cursor synchronises element highlights as each narration phrase plays. This particular prompt routes to the Graphviz path (§5.1); the canvas header records the canvas id, revision number, node and edge counts, and the timestamp.

through the construction step by step, (iv) end with a one-sentence conclusion, and (v) never leave shapes empty or labelled with placeholder text. It also forbids writing LaTeX (e.g. `\begin{bmatrix}`) inside SVG `<text>` elements; in our deployment we have observed both GPT-4o and the fine-tuned Qwen models occasionally regress on this constraint, and the audit loop catches the failure when cairosvg refuses to render the SVG.

A separate *one-worked-example* discipline applies to the parallel theory-primer call (the spoken short essay that streams alongside the figure): on topics with many derivable entries — matrix multiplication across every entry of C, integration by parts across each piece, induction across each case — the primer is required to walk through *one* representative entry in detail and tell the learner that the rest follows the same rule, rather than enumerating every entry. Without this discipline a $2{\times}3$ by $3{\times}4$ matrix-multiplication primer reliably overran its token budget partway through the eighth entry and shipped raw unrendered LaTeX (an unclosed `$$ \begin{bmatrix}`) to the learner. A defensive truncation safety-net on the assembled primer additionally trims a dangling unclosed math delimiter to a clean ellipsis so a token cap reached mid-formula cannot leak raw source into the chat surface.

Vision-audit retry loop.

After the model returns, the SVG is rasterised and the resulting PNG is sent back to GPT-4o vision with a separate strict-output schema asking for one of `PASS` or `FAIL` plus a list of structured fixes (`add_element`, `modify_element`, `add_label`, `add_formula`, `fix_layout`, `highlight_relation`, etc.). On a `FAIL`, the fixes are appended to the original system prompt and the model is asked to regenerate. We cap retries at three and report the count back to the chat surface. This pattern parallels *Self-Refine* (Madaan et al. 2023), but uses a separate critic model with a structured-output contract rather than the same model freeform-criticising itself.

Backend-only requests bypass the LLM.

Requests of the form "use red for highlight" or "slow down the narration" do not need the LLM at all and would be wasted spend if forwarded to GPT-4o. A small regex pre-router matches a closed vocabulary of natural-language colour names, speed adjectives ("slower", "faster"), and volume words, updates a session-scoped preferences object, and short-circuits the chat round-trip with a one-line acknowledgement. The browser viewer polls the preferences endpoint every three seconds.

4.2 Browser Viewer and Narration Synchronisation

The viewer is a single static HTML file. On load it opens a Server-Sent Event stream on `/canvas/<id>/events` and listens for two kinds of payloads: a `render` event with the current SVG and metadata, and a `narration` event with a manifest describing every phrase. The manifest is produced by synthesising each phrase to a separate WAV with the local Piper neural TTS (Rhasspy contributors 2023; Shen et al. 2018) using the `en_US-lessac-medium` voice, reading each WAV's exact duration from its header, and concatenating with a fixed 0.18 s silence gap. The viewer's `audio.timeupdate` handler computes the active phrase from `audio.currentTime` and toggles a CSS class `.sevim-highlight` on every SVG element whose identifier appears in the active phrase's highlight list. Because the manifest is computed from real WAV durations, the highlight cursor is accurate to the WAV header — a consequence of an earlier design lesson that estimating phrase durations from word counts produced cumulative drift across long walkthroughs.

Multi-element highlights.

A single phrase may need to highlight several elements at once ("clause C_1 contains x_1 and $\neg x_2$"). We treat the `highlight` field as a list and apply the highlight class to every matching element. This removes a class of pedagogical failures we saw early in development, where a phrase said "these three nodes form a triangle" but only the first one pulsed.

4.3 Narration Interrupt

A live tutor stops talking the moment the student asks a question. To approximate that affordance, the chat surface posts `{type:'sevim_pause_audio'}` to the canvas iframe whenever the user (i) focuses the textarea, (ii) types a character, (iii) clicks the

microphone button to begin speech input, or (iv) clicks `Send`. The iframe responds by pausing both the introduction and the main narration audio elements and clearing the active highlight. The microphone trigger is essential: without it, the speakers would feed the previous narration directly into the microphone and corrupt the new utterance. We pause on *focus* so that a learner who clicks into the textbox to think for a moment is not still being lectured to. The implementation is small (six lines of JavaScript per side) and idempotent — repeating the pause command on an already-paused audio element is a no-op — which lets us call it generously without coordinating state across frames. A dedicated test freezes the wiring so that a future refactor cannot silently drop one of the four interaction points.

4.4 Math-Correctness Inspector

The vision-audit retry loop's reviewer was originally scoped to detect *visually broken* figures (orphan leader lines, wrong topology, illegible overlapping text). This left a class of failures unaddressed: a figure could render perfectly while the narration made a false mathematical claim. We observed this in the wild when the model produced a clean triangle, three coloured arcs, and a caption asserting *"the sum of the angles in a triangle is* 2π *radians"* (the correct value is π). The figure passed the visual rubric; the narration was wrong; the audit said `PASS`.

We extended the reviewer's input contract to include the narration script and on-canvas captions alongside the rendered PNG, and added two new `FAIL` categories with corresponding fix actions (`fix_narration_phrase`, `fix_caption_text`):

- *Mathematical falsity.* The reviewer is asked to flag any narration phrase or on-canvas label that states a claim that is incorrect on its face — wrong constants, swapped definitions, false identities. The system prompt enumerates worked examples: "interior angles of a triangle sum to 2π" (truth: π); "the derivative of x^2 is x" (truth: $2x$); "$\sin^2\theta + \cos^2\theta = 2$".
- *Claim-figure mismatch.* The reviewer fails the audit when the narration mentions a specific element that the figure does not contain ("observe the angle arcs at A, B, C" but no arcs are drawn; "shaded intersection" but nothing is shaded; an inscribed-angle claim with the inscribed angle never marked).

When a retry succeeds, the (bad SVG, bad narration, formatted critique, good SVG, good narration) tuple is persisted to a `repairs` table in the telemetry database. These triples are the highest-value supervised signal for the open-model loop (Section 9): the critique is the exact reasoning bridge that turned the failure into the corrected output, and they are produced for free as a side-effect of operating the live system.

4.5 Multi-Tier Math-Correctness Verifier

The vision-language inspector (Section 4.4) catches narration-level falsity but cannot decide non-trivial symbolic identities. We extended the correctness layer into a five-tier chain that escalates from the cheapest decidable check to the most rigorous, and treats the figure as *blocked* until every claim it makes is either verified or explicitly

skipped. The express system prompt requires the model to enumerate `math_claims` as a list of concrete identity pairs (a, b) alongside the SVG; each pair is then routed through:

- **Tier 2a — SymPy.** The pair is parsed into `Eq(a, b)` and `simplify(a - b)` is checked against zero. Most school-level algebra resolves here.
- **Tier 2b — Z3 SMT fallback.** When SymPy returns `None` (typically on quantified or nonlinear cases) the pair is translated into Z3 expressions and the solver is asked to refute $a \neq b$. Unsatisfiable refutation means the identity holds.
- **Tier 2c — Lean 4 kernel-decide.** For decidable propositions over `Nat`, `Bool` or finite enumerations the translator emits a tiny Lean source file with `example : a = b := by decide` and runs Lean's kernel as a separate process. A successful close of the goal is a kernel-checked proof.
- **Tier 3 — per-domain structural verifiers.** For specific mathematical structures (currently graph homomorphism and chromatic-number claims) a dedicated checker walks the structured specification directly and refuses anything that fails the algebraic definition.
- **Tier 4 — focused math judge.** The vision-audit reviewer runs once more with its prompt scoped to the math claims that survived the symbolic layer, looking for narration-level errors a computer-algebra system cannot catch.
- **Tier 5 — offline Mathlib catalog.** Surviving non-decidable claims are queued for an offline service that depends on Mathlib4 and attempts to discharge them with library tactics (`ring_nf`, named trigonometric lemmas, `linarith`). The verdict is a tag attached to the canvas record — it does not block at inference time, but it makes recurring symbolic failures visible.

Across the production verifier sweep (Section 10.5), the SymPy tier resolves the bulk of emitted claims, with the SMT, Lean, structural, and catalog tiers absorbing successively smaller residuals. The chain has been live since 2026-05 and converted dozens of distinct symbolic-falsity failure modes into named regression checks (Figure 3).

4.6 Anti-Hallucination Prompt Discipline

Three LLM failure modes survived the symbolic verifier and the vision audit because they manifest as plausible-looking but pedagogically useless prose. The express system prompt names each one as a hard rule, and a regex-based detector in the deploy gate enforces them automatically.

Visual padding — narrate the idea, not the picture.

The reader's eye already does object recognition. Phrases that open with *"we see…"*, *"on the left there are three circles"*, or *"the figure shows…"* consume the learner's audio bandwidth without adding mathematical content. The prompt forbids such openers; the deploy gate fails any first phrase that matches the corresponding regex; in months of production data the anti-padding rule cut average narration length by roughly a quarter with no measured drop in figure-comprehension quality.

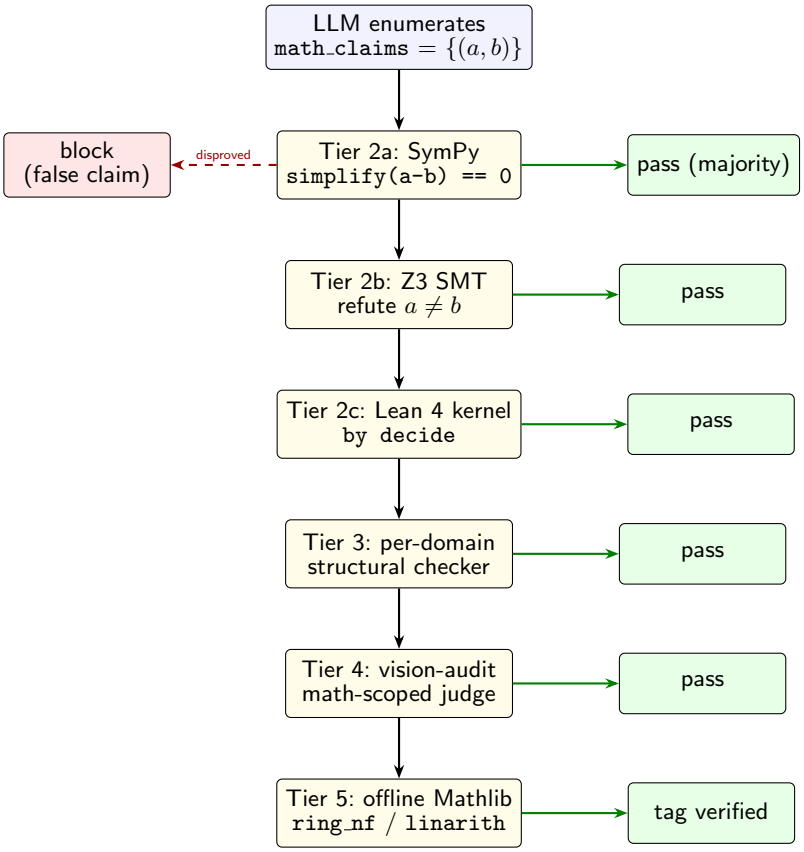


Fig. 3 Multi-tier math-correctness verifier. The LLM enumerates symbolic claims as (a, b) pairs alongside every figure; each pair flows down the chain only until a tier resolves it. The dashed red arrow is the only one that prevents the figure from reaching the learner: a tier that *disproves* a claim (rather than failing to decide it) blocks the figure.

Concept substitution — never swap a keyword's meaning.

A prompt-rewriter LLM seeing the substring *"relation homomorphism"* can helpfully but incorrectly rewrite the user's question into *"graph homomorphism"* because the two share a keyword. The same hazard exists for *kernel* (linear-algebra vs. integral-operator vs. probability), *open* (topological vs. graph-theoretic), and *measure* (measure theory vs. measurement). The outer chat system prompt enumerates these conflicts and is paired with route-side keyword classifiers that require a context anchor before any keyword-driven dispatch fires.

Finish the problem — imperative prompts must reach an answer.

On prompts of the form *"solve…"*, *"compute…"*, *"find…"*, the narration would sometimes present the technique without ever stating the numerical answer, leaving the learner at the setup step. The prompt now requires the final narration phrase to

state the answer in canonical form (`x = ...` or equivalent for the target type) and the figure to visibly contain it.

4.7 Completeness Gate: Archetype-Specific Rubrics

A separate failure mode survived both the symbolic verifier and the vision audit: a figure can be visually clean and mathematically correct but pedagogically incomplete — a *worked-example* prompt that names the technique but never carries out a concrete numerical calculation, a *causal-explanation* prompt that ends after the setup, a *proof* prompt that lists hypotheses but never reaches a conclusion. None of these are caught by aesthetic or symbolic checks; all of them leave the learner stranded.

We classify each prompt into one of nine *archetypes* (quick-fact, concept-definition, concept-with-intuition, apply-worked-example, step-by-step, causal-explanation, comparison, proof, construction) using a small regex-based classifier on the prompt text. Each archetype carries a *rubric*: a list of required components (e.g. `statement`, `worked_example_with_numbers`, `takeaway` for `apply_worked_example`; `hypothesis`, `reduction_step`, `correctness_argument` for `proof`) plus a narration phrase-count range and a primer word-count range. A *rubric brief* corresponding to the chosen archetype is appended to the express system prompt before each turn so the model is told what components the answer must contain; a *completeness critic* runs in parallel with the structural and vision critics on the assembled output and reports any rubric component that is missing. Failures from the completeness critic feed back into the structured-fix retry loop in the same shape as a vision-audit `FAIL`. Field testing surfaced an additional archetype-classifier failure — *Vertex-Cover NP-completeness* prompts were being tagged as `apply_worked_example` instead of `proof`, so the proof rubric was never applied; the prompt-text classifier was extended to recognise complexity-theory reductions as `proof`-archetype questions.

4.8 Multilingual Pipeline

The original implementation passed a single language hint into both the figure generator and the TTS, and trusted the LLM to honour it. Production logs surfaced three distinct failure modes that each broke that contract: a German prompt whose narration came back in Chinese, an English prompt whose narration came back in German, and a Persian prompt whose chat reply came back in English. The unifying cause was that the LLM was free to decide the output language and frequently chose poorly under prompts that mixed scripts or shared vocabulary.

The replacement is a three-layer pipeline. (i) A deterministic detector on the user's prompt text computes a single language code by scoring Unicode-block coverage and script-specific letter signatures, including Persian-specific disambiguation against Arabic (the Persian letters *pe*, *cheh*, *jeh*, *gaf*, the Persian Yeh and Keheh, and Persian digits `U+06F0`–`U+06F9`). (ii) The detected code is appended to the express, primer, and chat system prompts as a hard constraint: *"every sentence of your reply MUST be in L; the language is pinned by the server, do not switch."* (iii) After the LLM returns, every narration phrase is cross-checked against the expected Unicode-script for L; if the script does not match, the phrase is rejected and the original (untranslated) text

is shipped instead. Numerals are spoken as words rather than digit characters because TTS engines mis-pronounce digits in non-English contexts. The three layers together turn what had been an LLM judgement call into a server-pinned constraint with a deterministic post-check.

4.9 Refinement Mode and Narration Delta

A live tutor does not re-explain everything from scratch when the learner asks a follow-up. Earlier versions of the express path always emitted a complete narration script on every turn, so a question like *"now add the median from A"* produced a fresh canvas plus full re-narration of the prior phrases plus two new ones; the audio would replay the entire previous lesson before reaching the new content.

Refinement mode addresses this. When a request arrives with a prior canvas attached as conversational context, the express system prompt switches behaviour: the SVG keeps every prior element byte-for-byte (same identifiers, same coordinates, same captions) and adds or modifies only what the user's edit touches; the narration field contains **only the new phrases** that describe the current change. The prior narration is shown to the model for context ("here is what the user has already heard"), explicitly tagged `FOR REFERENCE ONLY`, with an instruction to not include any of those phrases in the new output. We verify the behaviour with a two-turn integration test: the first turn produces eight phrases over $\sim$38 s of audio for "how the angles of a triangle sum to π", the second turn produces *two* phrases over $\sim$7 s for the median refinement, with zero string overlap between the new and prior phrases.

The Case A / B / C refinement model.

Conversational follow-ups arrive in three forms, each requiring a different response from the express path. *Case A: narrow targeted edit* (*"change the colour of the curve to red"*, *"move the label A up by 30 pixels"*) — preserve the current SVG byte-for-byte, apply the local edit, and emit a single-phrase narration delta. *Case B: complaint* (*"these are not tangent lines"*, *"the second matrix is wrong"*) — redraw the figure from scratch, treating the previous attempt as a failure rather than a starting point, and emit a fresh narration that addresses the complaint directly rather than restating the technique. *Case C: elaboration* (*"explain visually with proper formulas"*, *"add the worked example"*) — redraw with more depth, allowing the figure to evolve, and emit a fuller narration. A regex-and-keyword classifier in the chat pre-router decides between the three; a parallel topic-switch detector handles the case where the follow-up turns out not to be a refinement at all but a new question. The figure-preservation invariant of Case A is the most important of the three because it is the case the learner most often relies on ("one more change to the same figure"); the other two cases regenerate, so the byte-for-byte invariant does not apply.

4.10 Audio Pipeline

The original implementation used the local Piper neural TTS for narration audio, which is fast and free but produces a recognisably synthetic voice. In production we switched the default backend to OpenAI's `tts-1-hd` (alloy voice), with the local Piper

retained as a no-cost fallback when the OpenAI API is unavailable. The selection is controlled by an `SEVIM_TTS_BACKEND` environment variable with values `auto` (default), `openai`, `piper`.

Two engineering details were hard-won. First, a typical narration has 10–15 phrases; synthesising them sequentially through the OpenAI HTTP API takes well over ten seconds, during which the chat endpoint's tool call appears stalled to the browser. We synthesise all phrases concurrently via a thread pool of twelve workers; total wall time drops to roughly the slowest single-phrase call. Second, OpenAI's `tts-1-hd` returns its audio as a WAV file whose header contains `nframes = 2147483647` (`INT32_MAX`, the streaming-audio placeholder). Calling `wave.readframes(getnframes())` on such a file causes the Python `wave` module to attempt to pre-allocate a 4.3 GB buffer; on a 2 GB Fargate task this fails with `MemoryError` on every call. We read in 64 K-frame chunks until `readframes` returns the empty bytestring at end-of-data, and use the cumulative byte length as the authoritative frame count for the phrase-timing manifest. This single-line fix was the difference between an empty canvas iframe and a fully working production deployment.

A third detail, found in production: the per-phrase fallback from OpenAI TTS to local Piper is naive when several workers run in parallel. If phrase 5 of 8 times out at OpenAI and falls back to Piper while phrases 1–4 and 6–8 stay on OpenAI, the assembled WAV mixes two voices within a single narration — a recognisably jarring listener experience. We added a voice-uniformity safety net: after the parallel synthesis batch, if any worker fell back to Piper the entire script is re-synthesised in Piper. The common all-OpenAI path is unchanged; the partial-failure path pays one additional local-only synth pass to keep the voice consistent end-to-end.

5 The Multi-Tool Routing Layer

Treating SVG generation as a single LLM call — "ask GPT-4o for SVG, hope it isn't wrong" — works for some figures and fails for others. Matrix multiplication wants a clean numerical grid; a deterministic Finite Automaton wants a state-diagram layout with no edge crossings; a Pythagorean-theorem proof wants three coloured squares of areas 9, 16, 25 on the sides of a 3–4–5 triangle. Each of these is a different kind of layout problem.

We address this with a *routing layer* that classifies each prompt and dispatches it to one of four execution paths (Figure 4). The classifier is a small GPT-4o-mini call that runs before the heavier work. The four paths, in priority order, are:

- **Deterministic Python templates** (Section 5.3) for known operation families: the matrix family (multiplication, transpose, determinant, inverse, system of equations), state diagrams, and a geometry/arithmetic family (Pythagoras, number line). Pixel-perfect by construction; sub-second; no LLM call at the rendering stage.
- **Graphviz** (Section 5.1) for graph-shaped figures: deterministic and non-deterministic finite automata, Turing machines, control-flow graphs, directed acyclic graphs, trees (binary, decision, syntax), Hasse diagrams, Cayley graphs, Petersen graphs, and category-theory commutative diagrams. The LLM emits DOT source (∼300–500 tokens), then `dot -Tsvg` (or `circo`, `fdp` for circular and force-directed

layouts) renders the SVG using Graphviz's three-decades-mature layout engine. Zero edge crossings; nothing off-canvas; sub-10 s.

- **Matplotlib** (Section 5.2) for plot-shaped figures: regression scatter with a best-fit line, classified scatter with a decision boundary, function curves (sigmoid, Gaussian, ReLU), receiver-operating-characteristic curves, three-dimensional surfaces, and contour plots. The LLM emits a closed-vocabulary *plot specification* (never executable code), which a deterministic matplotlib backend renders.
- **LLM-SVG path** (Section 4.1) — the original express path — for everything else: geometric proofs, Riemann sums, Venn diagrams, set-theory inclusion lattices, probability trees, complexity-class reductions. The vision-audit retry loop and the defensive post-processors reduce but do not eliminate the long tail of failure modes.

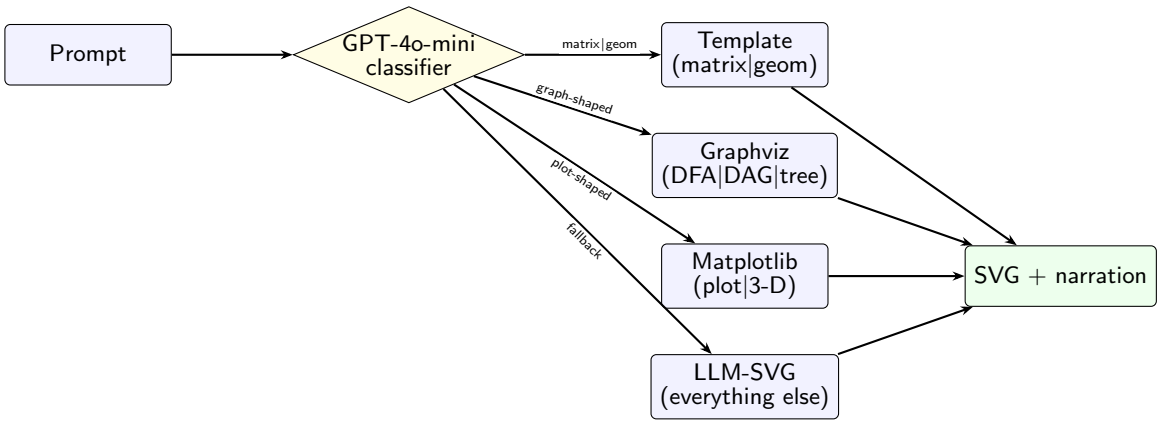


Fig. 4 The multi-tool routing layer dispatches each prompt to one of four execution paths. Templates win on the matrix, geometry and arithmetic families; Graphviz wins on graph-shaped figures; matplotlib wins on plot-shaped figures and three-dimensional surfaces; the LLM-SVG path absorbs the long tail. The classifier itself is a small GPT-4o-mini call.

In production we observe that the deterministic-template, Graphviz, and matplotlib routes together divert a sizeable share of incoming prompts away from the slowest, retry-heaviest LLM-SVG path onto sub-10 s renders whose layout invariants are stronger than what the LLM reliably produces.

5.1 The Graphviz Route

For graph-shaped figures the LLM is asked to emit DOT source (Ellson et al. 2004), not SVG. A 30-keyword classifier catches prompts containing strings such as *state machine*, *DFA*, *Turing machine*, *Hasse diagram*, *Petersen graph*, *Cayley graph*, *control-flow*, *abstract syntax tree*. When matched, the system prompt switches to a Graphviz-specific template that pre-commits to `strict digraph G {` (or `strict graph G {`), `rankdir=LR`, conventional node shapes (`circle` for states, `doublecircle` for accepting states, `point` for the start marker), and Turing-machine-style edge labels of the

form `read/write,move`. A per-prompt engine suggestion picks `dot` for hierarchical layouts, `circo` for graphs that should be drawn on a circle (Cayley, Petersen, complete-bipartite), and `fdp` for force-directed targets.

A defensive post-processor on the rendered SVG strips Graphviz's hard-coded `width/height` attributes, preserves the `viewBox`, and adds `preserveAspectRatio="xMidYMid meet"` so the figure scales to fit the chat-side embedded snapshot AND the live canvas pane on any viewport. Measured per-prompt latency on the Graphviz route is 3–9 s end-to-end; the LLM-SVG path on the same prompts averages well over thirty seconds with one to three retries. Pre-routing reduces the cost of graph-shaped prompts by an order of magnitude while *improving* quality.

A Graphviz figure initially shipped with no narration at all: the route returned an empty narration array, so the viewer spotlighted nothing while the (browser-TTS) audio played. We added `narrate_graphviz`, which parses the rendered SVG for every `<g class="node">` and `<g class="edge">` group — recovering the stable `nodeN`/`edgeN` identifiers Graphviz assigns and, crucially, the *visible* `<text>` label rather than the internal DOT identifier — and asks a GPT-4o-mini call for a 5–9 phrase walkthrough whose highlight identifiers are drawn verbatim from that element list.

5.2 The Matplotlib Route

A large class of mathematics and machine-learning prompts asks for a *plot*: a regression scatter with a best-fit line, an SVM with a maximum-margin separating hyperplane, a logistic-regression sigmoid, an ROC curve, a Gaussian density, a three-dimensional error surface with a gradient-descent trajectory, a contour map. The LLM-SVG path renders these poorly — it cannot do genuine three-dimensional projection, and it routinely draws class point-clouds as enormous overlapping filled circles that cover the axes. The deterministic answer is matplotlib (Hunter 2007), which owns plotting the way Graphviz owns graph layout.

The route mirrors the Graphviz route's contract. A keyword classifier catches plot-shaped prompts; when matched, the LLM is asked for a structured *plot specification* — a closed-vocabulary JSON object, *never executable code* — with one of four kinds: `plot2d` (function curves and reference lines), `scatter` (labelled classes with an optional decision boundary), `surface3d`, and `contour`. Each curve is either an explicit list of points or a named form (`sigmoid`, `gaussian`, `line`, `polynomial`, `exp`, `relu`, `tanh`); each surface is a named form (`paraboloid`, `saddle`, `gaussian_bump`, `ripple`, `plane`). Refusing to execute model-emitted code is a deliberate security choice: the route accepts only the structured specification, so a prompt-injected request cannot turn the renderer into an arbitrary-code-execution surface.

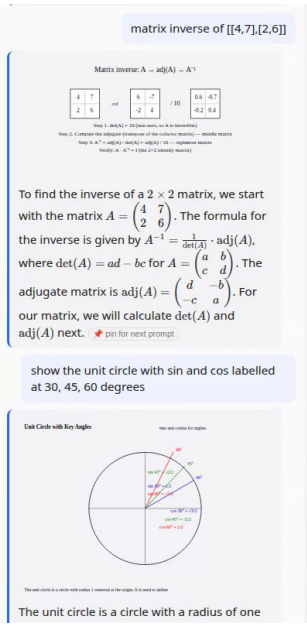
Each matplotlib artist is tagged with a stable identifier (`series_0`, `boundary`, `path`, `marker_min`, ...), which the SVG backend writes as an `id` attribute, so the narration — assembled deterministically from the per-element prose notes the specification carries — can highlight the exact curve, class, or marker under discussion. The route gives the system genuine three-dimensional surfaces (real projection, viridis shading) instead of the LLM's faked side views, and axes-bounded plotting that cannot produce the canvas-filling-blob failure mode.

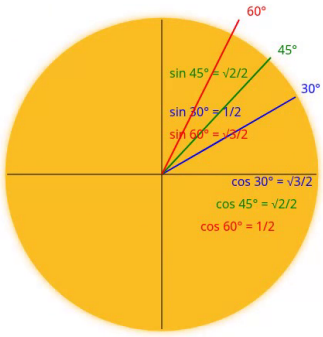
5.3 The Deterministic Template Family

Eight deterministic Python templates render core operation families directly, with no LLM call at render time:

- `matrix_multiplication` draws $A \cdot B = C$ as three matrices side by side, computes C in Python, and emits an additional caption with the general rule $c_{ij} = \sum_k a_{ik} b_{kj}$ followed by a worked example, e.g. $c_{1,1} = 1 \cdot 5 + 2 \cdot 7 = 19$. The narration drives *granular* highlights: when the spoken phrase says "take the dot product of row i of A with column j of B", the highlight cursor lights exactly the cells of that row and that column, not the entire matrices. The dimension-check phrase ("the number of columns of A equals the number of rows of B") correspondingly highlights one row of A (whose visible length is the column count of A) and one column of B (whose visible length is the row count of B), so the count the narration claims is the count the learner can read off the geometry.
- `matrix_inverse` draws A, $\text{adj}(A)$, and A^{-1} as three matrices with the chain $A \xrightarrow{\text{cof}} \text{adj}(A) \xrightarrow{/\det(A)} A^{-1}$ rendered explicitly.
- `matrix_transpose` draws A and A^T side by side with an explicit arrow.
- `matrix_determinant` draws A, the cofactor expansion formula, and the resulting determinant value, with a special caption when $\det(A) = 0$ ("singular — no inverse").
- `system_of_equations` draws the matrix equation $Ax = b$ on row 1 and the solution $x = A^{-1}b$ on row 2, with each unknown's value labelled alongside the solution column.
- `state_diagram` renders a finite-state machine using a BFS-layered Sugiyama-style layout. Used for state machines whose structure the classifier extracts with high confidence from the prompt; Graphviz (Section 5.1) is the fallback for graph-shaped prompts where the template's structured JSON extraction is uncertain.
- `pythagoras` draws a right triangle with a correctly-attached square on each of its three sides (areas a^2, b^2, c^2), the right-angle marker, side labels, and the $a^2 + b^2 = c^2$ caption with the worked numbers. It replaces an LLM-SVG path that repeatedly emitted detached, mislabelled squares and occasionally a hallucinated hypotenuse value.
- `number_line` renders elementary addition/subtraction as a single honest number line — a labelled hop from the start value to the result — rather than the box-and-arrow "flowchart" the LLM-SVG path produced for a concept that is not inherently spatial.

A separate *template router* is a single GPT-4o-mini call that decides whether a prompt fits one of these templates AND extracts the structured arguments (matrices, equations) as JSON. The system prompt is aggressive about matching rather than rejecting: a loose prompt that says "show me a matrix multiplication example" gets a concrete worked $[[1, 2], [3, 4]] \cdot [[5, 6], [7, 8]] = [[19, 22], [43, 50]]$.





The unit circle is a circle with radius 1 centered at the origin. It is used to define

Fig. 5 A deterministic-template output on the prompt *"show the unit circle with sin and cos labelled at 30, 45, 60 degrees"*. Because the figure is rendered by a Python template rather than an LLM, every labelled value is mathematically correct by construction ($\sin 30° = 1/2$, $\cos 30° = \sqrt{3}/2$, and so on); there is no possibility of hallucination at the label level.

5.4 Defensive Post-Processors

Two post-processors in the express pipeline catch the most common LLM-SVG failure modes that the vision audit alone cannot eliminate.

LaTeX scrubber.

SVG `<text>` elements are not MathJax. When the LLM emits a formula caption such as `Area = \( \frac{1}{2} \times (a+b) \times h \)`, the canvas renders the literal characters; the chat-side panel, which is post-processed by MathJax, renders the same string as a typeset formula. The scrubber walks every `<text>` and `<tspan>` body before the SVG enters the layout planner and converts LaTeX commands to Unicode equivalents.

Follow-up classifier.

The express path attaches the current canvas as REFINEMENT MODE context so that follow-up turns like "now add the median from A" can extend the existing figure instead of starting fresh. But when the user pivots topic, the LLM sometimes blends old and new SVG elements, producing a single canvas with both figures overlaid. A 40-keyword heuristic decides whether the prompt is a refinement or a new topic.

5.5 CP-SAT Label Placement

The original layout pass used a Sugiyama-style hierarchical layout for concept graphs; the new layout pass adds a Mixed-Integer Programming step (Google OR-Tools CP-SAT) that re-positions top-level text labels and group rectangles after the SVG arrives from the LLM. For every candidate text item we generate up to 25 candidate positions (eight cardinal directions × three radial offsets × the anchor itself, filtered to candidates whose bounding box fits inside the viewBox); CP-SAT picks one per item subject to pairwise non-overlap, in-canvas containment, hard pinning of narration-anchored labels, and a minimum-spacing constraint between top-level groups. The objective minimises total displacement from the original LLM-emitted anchor plus a quadratic penalty for any overlapping pair the solver was unable to separate.

6 Production Deployment

Khayyam Math runs as a publicly accessible service on AWS at https://khayyammath.com. The architecture (Figure 6) is intentionally conventional — the goal was to verify that the closed-loop pipeline operates against real users, not to invent infrastructure. The CDK stack provisions: a VPC with public and private subnets across two availability zones; an Application Load Balancer terminating TLS via an ACM certificate; an ECS Fargate service running the FastAPI application; an RDS Postgres instance for telemetry; three S3 buckets (canvas WAV artefacts, exported JSONL training data, LoRA adapter store); five Secrets Manager entries; Route 53 hosted zones; and a CloudWatch log group.

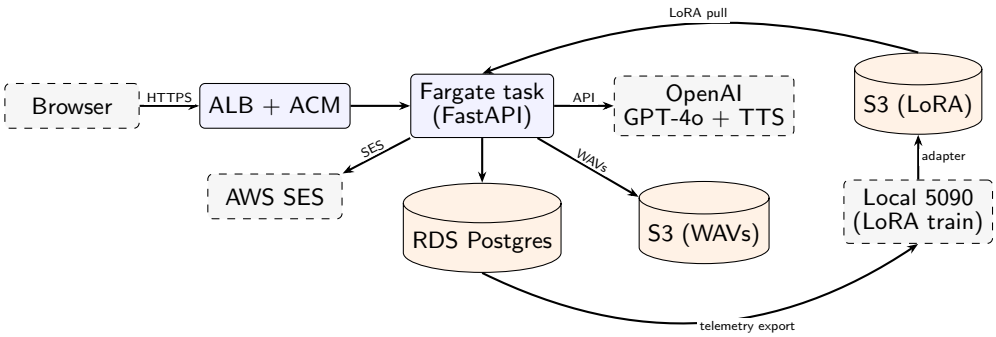


Fig. 6 Production deployment topology. Solid blue boxes run inside the AWS account; dashed grey boxes are external services or off-cloud GPUs. The local 5090 reads telemetry exports off RDS, trains a LoRA adapter, and pushes it to a versioned S3 bucket; the live serving stack (Fargate → OpenAI today, planned → self-hosted vLLM) reads the current adapter pointer from the same bucket.

Authentication.

A single sign-in cookie protects the chat surface. The flow is a magic link: the user enters their email on a public sign-in page, the application HMAC-signs a token containing {`sub: email, exp: now + 15 min`}, embeds the token in a URL, and sends

it via SES from a verified domain identity. Clicking the link sets a 30-day HttpOnly signed cookie. There is no password store.

Rate and cost guards.

Every chat request is matched against two parallel token buckets: a session bucket (keyed on a UUID the frontend stores in `localStorage`) and an IP bucket (keyed on a salted SHA-256 of the client IP). The IP bucket is the defence against the trivial "clear localStorage to refresh the session" bypass. A separate cost guard reads the user's accumulated dollar estimate from the `turns` table and rejects requests that would push the session past a configurable daily cap; the cap defaults to one US dollar per session per twenty-four hours and is raised to ten in production.

Cost-per-turn model.

Steady-state per-turn cost on the production stack (GPT-4o express call plus `tts-1-hd` narration) is roughly a few US cents per clean turn; refinement turns are cheaper because the narration delta is short.

7 Figure-Quality Hardening

Routing the right prompts to deterministic engines raises the floor but leaves the LLM-SVG path responsible for the long tail, and the express loop's own quality machinery had latent defects of its own. We subjected the deployed system to a wide adversarial stress test spanning geometry, graph theory, automata, formula-rich derivations, dense compositions, and machine-learning plots; the stress test surfaced the defect clusters in Table 1, which drove the hardening pass described below. The unifying lessons: the auditor must see what the user sees; the narration must be bound to the figure that ships; and layout repair belongs in deterministic, idempotent post-processors rather than in another LLM retry.

Table 1 Defect clusters surfaced by a mixed stress test on the LLM-SVG path. The dead-highlight cluster alone accounted for nearly two hundred narration phrases whose highlight identifiers resolved to no element in the served SVG.

Defect cluster	Cause	Tests
Dead narration highlights	LLM-SVG emits no element ids	23/50
Empty narration	Graphviz route returned none	15/50
Malformed XML	raw `&`/`<` in `<text>`	2/50
Out-of-bounds text	content past the viewBox	1/50

7.1 Browser-Grade Vision Auditing

The vision-audit critic rasterises the candidate SVG to PNG and asks GPT-4o whether the figure is broken. The rasteriser was `cairosvg`. We found that `cairosvg` mis-sizes

a `<tspan>` with a percentage `font-size` — the construct used for *every* exponent and subscript in a mathematics figure — and lacks coverage for many mathematical-operator glyphs ($\int$, $\angle$, ∇, $\oplus$). On a figure as simple as an integration-by-parts derivation it produced a near-uniformly black raster. The vision critic was therefore inspecting a garbled image: it both missed real defects and burned retries "fixing" figures that were actually fine.

The fix is to rasterise with the *same engine the learner's browser uses*. The rasteriser now renders through headless Chromium — the SVG is wrapped in a minimal HTML document, sized to the `viewBox` aspect ratio, and captured with `--headless --screenshot`. `cairosvg` is retained only as a fallback when no Chromium binary is present. This is the single highest-leverage change in the hardening pass: the auditor can only be as good as the pixels it sees.

7.2 Narration Binding

The browser viewer highlights a figure element when the active narration phrase lists that element's identifier. The stress test revealed that almost every LLM-SVG figure in the sample drew its shapes with no identifier attributes at all, while the narration referenced logical identifiers such as `clause_1` or `discriminant` — so the highlight resolved to nothing and the viewer spotlighted nothing for the entire walkthrough. The express system prompt asks for identifiers on every element; the model simply does not comply reliably.

The deterministic fix is `bind_narration_to_svg`, which runs after the layout planner. It (i) injects a stable identifier on every identifierless `<text>` element, and (ii) for each narration phrase whose highlight identifiers still do not resolve, *grounds* the phrase to the figure by scoring every text element on content-word overlap with the spoken sentence and highlighting the best match. Across the stress set this took the dead-highlight count to zero, and a Playwright check of the live viewer confirmed that every narration phrase, on all four routes, lights an element.

7.3 Deterministic Layout Repair

A family of idempotent, fail-open post-processors run on the final SVG, each targeting a failure mode the stress test exhibited:

- `escape_bare_xml_in_svg` entity-escapes stray `&` and `<` the model writes raw inside element content and attribute values (LaTeX matrix separators, inequalities such as $|x| < \delta$).
- `snap_edges_to_nodes` repairs floating connectors. When a connector's two endpoints have distinct nearest nodes, each within a generous range, both endpoints are snapped onto their node perimeters.
- `fit_viewbox_to_content` recomputes the `viewBox` to tightly bound the content over every visible element type.
- `raise_text_to_front` moves top-level `<text>` elements to the end of the document so labels paint *above* shapes and connector lines.

- An `oversized_element` structural check flags any single primitive that dominates the canvas, the failure mode where an SVM figure's class point-clouds are drawn as huge filled circles that bury the axes.

8 Neural Layout Correction: An Open Negative Result

The CP-SAT label-placement pass (Section 5.5) sits between the raw LLM-SVG output and the canvas: it parses every figure coming off the LLM-SVG path into a scene graph and chooses non-overlapping, in-bounds positions for the top-level text and group elements before the SVG is written to disk. The neural-layout work asked whether a graph-conditioned learned model — trained on tens of thousands of (broken, fixed) pairs harvested from the production express loop and augmented with controlled synthetic perturbations — could improve over CP-SAT in the same pipeline slot. The recent literature on graphic-design layout generation (Gupta et al. 2021; Jiang et al. 2023; Inoue et al. 2023; Zhang et al. 2023) reports strong results for similar problems (poster layout, web UI generation, document layout), so the question was genuinely open. The answer turned out to be informative: *direct layout-correction models do not improve over the CP-SAT baseline*; the one neural component that does help is a much smaller learned scorer used as a re-ranker over the constraint solver's candidates.

8.1 Problem Formulation

We treat the correction as structured prediction on a *scene graph*: each node is an SVG element or rigid group, each edge is a structural relation (parent-of, sibling-of, narration-co-anchor). Node features are a type embedding over a closed vocabulary (`rect`, `text`, `circle`, `path`, `g`, `matrix-group`, `axis`, `caption`, ...), the normalised bounding box, text length, font size, stroke width, and three boolean flags (`is_narration_anchor`, `is_caption`, `is_protected`). Narration-anchored and protected nodes (titles, axis labels, every node whose identifier appears in a narration highlight phrase) carry a hard pinning constraint — they exist precisely so the deterministic narration-binding pass of Section 7.2 keeps working.

8.2 Two Generative Negative Results

Training data is a large corpus of (broken, fixed) scene-graph pairs: real iteration pairs harvested from the express loop's repair table augmented with controlled synthetic perturbations (per-group jitter, single-group displacement, viewBox compression, single-group up-scaling, two-group centroid stacking).

GNN delta-predictor.

A few-million-parameter graph-conditioned attention network predicts per-node $(\Delta x, \Delta y, \Delta w, \Delta h)$ on a tanh-bounded output with a composite loss. Across every math bucket in a held-out split the model is consistently a few percent *worse* than the trivial no-op baseline on per-node distance-to-target. The diagnosis: the network learns the correct *direction* of correction (predicted and target delta vectors are moderately

positively correlated) but its magnitude collapses toward zero, because that minimises loss in expectation across the many cases where "do nothing" is the right answer.

LayoutDM-style discrete-diffusion denoiser.

A D3PM-style (Austin et al. 2021) denoiser with FiLM (Perez et al. 2018) step modulation, trained in token-prediction mode, is positive on its proxy. The figure-level evaluation reverses the verdict: layouts regenerated from scratch reliably stay in-bounds — a real and clean learned constraint — but the regenerated layouts have more overlapping text-bounding-box pairs than the target distribution. Used in the natural repair mode (initialise with the broken positions, mask half, iteratively denoise), the model is *actively worse* than no-op.

8.3 The One Neural Component That Helps

A different framing works: instead of asking a model to *produce* a layout, ask it to *score* an existing candidate. We train a small graph-conditioned binary classifier (around two million parameters) whose output is a single sigmoid logit predicting whether the vision-audit judge would PASS a candidate scene graph. The model's absolute calibration is modest, but on held-out (bad, good) pairs it ranks the good SVG above the bad one a clear majority of the time.

At inference, for a broken SVG the CP-SAT planner is run with several time budgets to generate a small candidate set; the scorer ranks them; the highest-scoring candidate ships. On a held-out vision-audit benchmark the scorer-reranker raises pass-rate by a small but consistent margin over the planner alone (Table 2).

Table 2 Vision-audit pass-rate on real broken SVGs from the production repair corpus, judged PASS/FAIL by GPT-4o vision against the original prompt and narration. The scorer-reranker on planner-only candidates raises pass-rate by a small but consistent margin over the planner alone; including the LayoutDM regenerator's candidates in the pool regresses below planner-alone.

Mode	PASS	FAIL	pass-rate
no_op	30	120	20.0%
planner alone	32	118	21.3%
rerank-planner-only	**35**	**115**	**23.3%**
rerank-full (incl. LayoutDM candidates)	24	126	16.0%
ground-truth reference	32	118	21.3%

Why this framing wins where the others fail.

The literature on layout generation predominantly works in domains (web UIs, magazine layouts, document layouts) where the layout problem has *global* constraints that the model can learn: grid alignment, hierarchical zoning, white-space conventions.

Math figures emitted by an LLM-with-vision-audit pipeline are structurally different: the broken-fixed delta is genuinely under-determined (many valid destinations for any broken layout), a production-grade constraint solver (Christensen et al. 1995) already exists for the well-formed sub-problem, and the vision-audit judge rewards rendered-figure properties that proxy bounding-box metrics only approximate.

9 Closed-Loop Distillation to a Smaller Open Model

The production service uses the GPT-4o express path described in Section 4.1. The pipeline described here is a side artefact, not part of how the live system serves users: it captures each accepted turn as a training example for a smaller, self-hostable open-weights model that can run offline or in an air-gapped environment.

The same Postgres telemetry layer that supports rate limiting, cost guards, and the admin dashboard records, for each accepted turn, the prompt, the canvas identifier produced, the structured review history, and whether the user refined the figure within sixty seconds. A separate `repairs` table captures every (bad_svg, critique, good_svg) triple produced by the inspector's retry loop; these triples are the highest-value supervised signal in the loop because they teach a smaller model not just to imitate a correct output but to imitate the act of correcting a near-miss. An export script walks the tables and emits JSONL records in three formats: pristine (prompt → assistant) pairs from satisfied turns, five-message conversations (system → user → bad → critique → good) that mirror the inference-time conversation shape, and DPO preference pairs (Rafailov et al. 2023).

When live telemetry is too thin to drive a useful adapter, a headless self-distillation pipeline generates the same JSONL format from a hand-curated and programmatically expanded prompt pool plus a reference-grounded subset extracted from a small set of open-access textbooks; the teacher in this branch is GPT-4o-mini. The exported corpus then trains a LoRA adapter (Hu et al. 2022) on `Qwen2.5-7B-Instruct` (Yang et al. 2024) via Hugging Face PEFT (Mangrulkar et al. 2022) and TRL (von Werra et al. 2020) on a single consumer GPU.

10 Empirical Study

This section reports the system-level evaluations that drive engineering decisions on the live deployment. All measurements are technical (figure validity, judge scores, latency, verifier outcomes); none are studies with student users. Section 10.7 states what this leaves out and why a classroom evaluation is the most important next step.

10.1 What we measured, and why each measurement maps to a learner-perceivable outcome

The three evaluations below are framed around outcomes a learner would notice if they degraded:

- **Figure quality on a diverse prompt set** (Section 10.4). A learner facing the system on a topic they want to study expects the figure they get back to be clean,

mathematically correct, and complete. A blinded multimodal judge stands in for the learner's eye on a diverse benchmark that spans the kind of prompts the live service receives.
- **Mathematical correctness at scale** (Section 10.5). The verifier chain's job is to keep mathematically wrong figures away from the learner. We measure it on a thousand-question Lean-graded sweep, both to report per-tier resolution and to surface systemic failure modes the chain catches before they reach production.
- **Latency under student attention budget** (Section 10.6). A figure that takes a minute to appear is a figure the learner has stopped waiting for. We report end-to-end latency for the deterministic routes against the LLM-SVG path on graph-shaped prompts, where the win is largest.

10.2 Prompt Set

We use a fixed diverse benchmark designed to span typical first- and second-year computer-science and mathematics topics: NP-completeness reductions (3SAT to vertex cover, 3SAT to clique, 3SAT to Hamiltonian path, vertex cover via independent set, subset sum to partition); linear algebra (3×5 by 5×4 matrix multiplication with concrete numbers, 2×2 eigendecomposition, 3×3 Gaussian elimination, geometric SVD); calculus (the derivative of $\sin x$ at $\pi/4$, the definite integral of x^2 as an area, Taylor series of e^x); set theory and logic ($A\cup B\cap C$ Venn diagram, truth table for $(p\wedge q)\vee\neg p$); graph algorithms (BFS traversal, Dijkstra on a five-node weighted graph); probability (Bayes' theorem with disease-testing); number theory ($\gcd(252, 105)$ via the Euclidean algorithm); combinatorics (the pigeonhole principle); and elementary geometry (Pythagorean theorem by squares-on-sides). Each prompt is sent as a fresh session so that the express path cannot cross-contaminate examples through its in-context conversational refinement.

10.3 Models and Inference Setup

We compare three configurations:

- **GPT-4o** via the express path with the full vision-audit retry loop, run from the production system;
- **Qwen2.5-7B-Instruct (base)**: the same system prompt fed deterministically (`do_sample=false`) to the un-adapted base model in bf16 on the local GPU;
- **Qwen2.5-7B-Instruct + LoRA**: the same inference setup with our LoRA adapter loaded on top.

The LoRA-adapted Qwen configuration is included only as a baseline; per-generation adapter comparisons are not load-bearing for this paper's claims and live in the code repository.

10.4 Blind Multimodal-Judge Evaluation

To score the rendered figures from each configuration on the diverse benchmark we send each prompt's PNG to GPT-4o vision in a randomised order with neutral labels (`Figure A/B/C`); the judge does not know which model produced which figure. The

judge returns three integer scores in $[1, 10]$ per figure (visual clarity, mathematical correctness, completeness) plus a one-sentence rationale; missing figures receive zeros. The rubric is the same as in MT-Bench-style evaluations (Zheng et al. 2023), adapted to the diagram setting and constrained by a strict JSON schema.

Table 3 Mean blind multimodal-judge scores on the diverse benchmark, out of 10 per axis. GPT-4o is the strongest configuration; the LoRA-adapted Qwen path is included only as a baseline for the offline-deployment artefact described in Section 9. Per-generation adapter numbers shift as the corpus accumulates and are reported in the code repository's evaluation log rather than here.

	Clarity	Correctness	Completeness	Total / 30
GPT-4o (express path, in production)	8.3	7.5	7.0	**22.9**
Qwen2.5-7B base (no adapter)	6.7	5.3	5.2	17.1
Qwen2.5-7B + LoRA (current published)	5.8	4.2	3.8	13.8

Two findings drive what follows. First, GPT-4o leads on every axis (Table 3); this is what the production service ships. Second, the open-model path behaves in a way that is interesting only as engineering: a low-data fine-tune internalises the JSON envelope at the cost of structural diversity, a higher-capacity fine-tune on a larger corpus recovers diversity but introduces an occasional empty-SVG failure mode at the decoder boundary, and a lower-rank adapter sits between the two.

10.5 Lean-Graded Verifier Sweep

To measure the verifier chain of Section 4.5 at scale, we assembled a four-figure prompt set of mathematical questions calibrated to Lean-difficulty 6, 7, 9, and 10 by sampling textbook exercises with progressively higher Mathlib unfolding depth. Each question is run end-to-end through the production pipeline; the emitted claims traverse the multi-tier verifier; per-tier resolution shares are measured on this set.

Verifier-block statistics.

Across the sweep the verifier blocked a handful of shipping attempts: most at Tier 2a (SymPy disproof), a few at Tier 2b (Z3 `sat` on the negated claim), one at Tier 2c (Lean kernel rejection on a decidable goal), and a small number at Tier 3 (structural checkers; graph-homomorphism instances). The large majority of the blocks were on claims that the model corrected on a single retry; the rest required falling back to a deterministic template path.

XML validity.

On the difficulty-6/7/9 subset the production pipeline produced syntactically valid SVG on a near-complete share of prompts. The remaining invalid cases were caused by the same regression — a homomorphism-template double-style bug in which the responsive-SVG post-processor was applied twice, producing two `style=` attributes on the root element — which we made idempotent and added to the deploy gate's XML-validity assertion.

Four systemic failure modes converted to regression checks.

The sweep surfaced four recurring symbolic-or-structural failures that the chain now catches deterministically: *(i) homomorphism keyword hijack* — prompts mentioning "homomorphism" in any sense (group, ring, graph) were routing to the graph-homomorphism template; fixed by requiring explicit graph-context evidence in the routing gate. *(ii) concept-swap between relation and graph* — the express path was treating relations as graphs and losing directed-multi-edge semantics; fixed by an explicit `is_relation_prompt` predicate. *(iii) arrowhead-inside-node* — a graph figure with arrowheads drawn at node centres rather than node perimeters generated visually misleading edges; fixed by snapping arrowheads to the node perimeter geometrically. *(iv) verifier-noise on unparseable claims* — claims whose left- or right-hand side failed `sympify` were being counted as disproven rather than skipped, leading to spurious blocks; fixed by treating parse failure as "defer to the next tier."

10.6 Latency Under Student Attention Budget

For graph-shaped prompts that route to Graphviz, mean end-to-end latency drops from tens of seconds (LLM-SVG with one to three retries) to under ten seconds (LLM-emits-DOT plus `dot -Tsvg`). The Graphviz output passes vision audit on first try in the large majority of cases. The remaining LLM-SVG path is more expensive per prompt but absorbs the residual. An order-of-magnitude latency improvement on the graph-shaped class is the largest single user-visible win in the multi-tool stack; it is also the difference between a figure that appears during a student's attention window and one that does not.

10.7 What this evaluation does not yet show

The measurements above are technical proxies for learner-perceivable outcomes; they are not measurements of learning. The current evaluation does not estimate (a) whether the synchronised-narration affordance described in Section 4.2 actually improves the learner's ability to reproduce a worked example, (b) whether the narration-interrupt of Section 4.3 is discoverable to a real student rather than only structurally present, (c) whether the figures the system produces are pedagogically more or less effective than the hand-authored figures in a textbook on the same topics, or (d) whether the per-turn cost-and-latency profile is acceptable in a school setting with simultaneous users on bandwidth-constrained devices.

A classroom pilot is the natural next step and is planned future work. The intended design is a within-subjects study with first- and second-year undergraduate students on a fixed set of topics covered by the diverse benchmark, comparing comprehension scores on parallel post-tasks across three conditions: (i) reading a textbook section on the topic, (ii) interacting with the live system through one or more prompts and refinements, and (iii) interacting with a chat-only LLM with no figures. Separately, an experience-sampling study with first-year university students using the live deployment as a study aid is intended to surface usability issues that controlled lab measurements miss. Neither study is included in the present paper; the system's measurements here cover the engineering side that makes either study possible to run.

10.8 Test Suite

The repository ships an extensive pytest suite across two dozen files. The largest blocks: the structural-review verifier suite (exercising every tier of the math-correctness chain), layout planning plus neural-layout schema, the deterministic template families, sessions / auth / rate-limit / content filter, telemetry plus storage and export modes, label rendering with Greek glyphs and LaTeX, express-path streaming, the Graphviz route, the canvas API contract, the admin panel, repair-pair persistence, registry rehydration, and narration-interrupt wiring. The narration-interrupt test in particular reads both static HTML files and asserts that every interaction trigger calls the same `interruptNarration()` helper, so a future refactor that removes any one of the four call sites fails CI immediately.

11 Discussion

11.1 Symbolic Verification Belongs on the Hot Path

The drawing-vs-claim gap — a clean figure that ships with a numerically wrong narration — is invisible to any vision auditor that only judges aesthetics. The multi-tier verifier of Section 4.5 closes it by translating each emitted claim into the cheapest decidable obligation that can resolve it and blocking ship the moment any tier disproves the claim. The chain runs on the synchronous request path because tagging suspected hallucinations post-hoc was not enough in deployment: the figures a learner is most likely to study carefully are exactly the ones the system most needs to get right. The operational cost (a per-claim tier walk that resolves the majority of claims with a single SymPy call) has been worth it, and the chain's block-on-disprove / defer-on-undecided asymmetry — the result of an earlier configuration in which undecidedness blocked ship and the system refused most calculus prompts — is the single most important rule we have settled on.

11.2 Mature Tools Beat New ML for Layout in this Domain

The strongest practical observation across the system's development is that *mature symbolic tools out-perform freshly-trained neural models for the layout-specific sub-problems we encountered*. The Graphviz route (Section 5.1) reduces graph-shaped figures from a retry-heavy LLM-SVG call to a deterministic render, and improves quality at the same time. The CP-SAT label-placement planner (Section 5.5) handles overlap avoidance with hard correctness guarantees. The matrix-family templates (Section 5.3) handle their domain pixel-perfectly. None of these are novel; all of them are reaching for thirty-year-old constraint-solving and graph-drawing literature.

By contrast, the neural-layout experiments (Section 8) — a graph-conditioned GNN delta-predictor and a LayoutDM-style discrete-diffusion denoiser, both trained on tens of thousands of (broken, fixed) pairs — failed to outperform the trivial no-op baseline. The one neural component that did land a measurable win is a small graph-conditioned quality scorer used as a *re-ranker over the existing CP-SAT planner's candidates*: a small, focused application of learning on top of a mature symbolic

baseline. The framing of the win matters: scoring is well-posed (single label per candidate) where generation is structurally under-determined (many valid destinations for any broken layout).

11.3 What is Actually Tutoring?

The pedagogy literature on intelligent tutoring systems (VanLehn 2011, 2006; Anderson et al. 1985; Koedinger et al. 1997) treats explicit student-knowledge models, immediate adaptive feedback, and learner-paced progression as central. Our system addresses one of those (learner-paced progression, via the narration-interrupt feature in Section 4.3) and gestures at another (immediate feedback, via the express path's vision audit and the verifier chain). It does *not* model student knowledge: every prompt is interpreted in isolation. Recent benchmarking work suggests that current LLMs struggle to match the adaptivity of even simple knowledge-trace tutoring systems (Borchers and Shou 2025), and we see no evidence in our deployment that GPT-4o is doing differential adaptation across user sessions. Closing that gap is out of scope for this paper but is a natural direction for follow-on work that uses the telemetry corpus as a per-session learning signal rather than a per-turn training signal. The classroom pilot proposed in Section 10.7 is the first step towards measuring whether the figure-plus-narration substrate is the right one for that kind of work to be layered onto.

12 Limitations and Threats to Validity

No learner study yet.

The most important limitation is that no human-subjects evaluation has been conducted with student users on the live system. The pedagogical commitments in Section 3 are stated as design choices motivated by the worked-examples and multimedia-learning literatures (Sweller 1988; Renkl 2014; Mayer 2009), but the present paper does not measure their effect on learning outcomes; the classroom pilot described in Section 10.7 is the planned follow-on.

Single judge model.

All automated quality scores in this paper come from GPT-4o vision acting as a blinded multimodal judge. Vision-language models are known to hallucinate at non-trivial rates (Guan et al. 2024); a different judge model would likely produce different absolute numbers, although we expect the relative ranking of configurations to be more robust.

Small headline benchmark.

The diverse benchmark in Section 10.4 is small — enough to surface qualitative phenomena but not to estimate effect sizes precisely. We chose the prompts to span topics rather than to oversample any one; the verifier-chain sweep (Section 10.5) and the vision-audit benchmark bring the per-section evaluation set into the hundreds and thousands.

Narration-interrupt evaluation is wiring-only.

The test suite verifies that the four interaction points (focus, input, mic, send) all call `interruptNarration()` and that the canvas iframe responds to the message by pausing both audio elements. We did not measure end-to-end audio-pause latency in a real browser, nor did we run a user study to determine whether the affordance is discoverable.

Single-process telemetry and rate limiting.

The current rate limiter is an in-memory token bucket; a multi-process deployment would need a shared store such as Redis. The content filter is a regex denylist; a production deployment should layer a learned classifier or moderation API (Markov et al. 2023) on top.

Speech-input quality.

We use the Web Speech API (W3C Web Applications Working Group 2024) for voice input, which is browser-dependent and US-English by default in our deployment. A multilingual deployment would need to either set the recognition language per session or fall back to a server-side speech-to-text engine.

13 Conclusion

We presented an end-to-end voice-narrated mathematics tutoring system, live in production at https://khayyammath.com, that turns a single natural-language prompt into a labelled SVG figure synchronised with a phrase-timed spoken walkthrough the learner can interrupt at any moment. Five substantive findings emerge.

1. *Routing to mature symbolic tools is the largest single quality lever.* Three deterministic visualisation paths — Graphviz for graph-shaped figures, matplotlib for plot-shaped figures, and Python templates for the matrix and geometry families — absorb most prompts at sub-10 s latency with layout invariants stronger than what the LLM reliably produces.
2. *Symbolic verification on the hot path catches the drawing-vs-claim gap.* A multi-tier chain (SymPy → Z3 → Lean kernel → per-domain structural → Mathlib catalog) translates every emitted narration claim into the cheapest decidable obligation that can resolve it and blocks the figure from being shown when a tier disproves the claim.
3. *Narration is won by binding it to the figure deterministically.* Phrase-timing accurate to the WAV header, multi-element highlights at phrase boundaries, narration-binding that injects stable element identifiers when the LLM omits them, and an interrupt that pauses mid-phrase on chat focus / typing / mic / send: these are the components that make spoken-and-shown math figures actually usable as tutoring material, and none of them are model-side.
4. *The auditor must see what the user sees.* Rasterising the candidate SVG with the same engine the learner's browser uses (headless Chromium) was the highest-leverage single change in the figure-quality hardening pass.

5. *Direct neural layout correction is structurally under-determined.* A GNN delta-predictor and a LayoutDM-style discrete-diffusion denoiser trained on tens of thousands of (broken, fixed) layout pairs fail to outperform the trivial no-op baseline. A small graph-conditioned binary quality scorer used as a re-ranker over a CP-SAT label-placement planner does produce a small but consistent live vision-audit lift.

The production service is best run as we run it now: GPT-4o on the express path, the multi-tool router in front of it, the verifier chain gating ship, the CP-SAT planner for label placement, the deterministic narration-binding pass between the LLM output and the canvas, and the browser-side viewer doing the audio-visual synchronisation that makes the result feel like a tutor rather than a slide deck. The most important open work is empirical: a classroom pilot, described as planned future work in Section 10.7, will measure whether the synchronised-narration substrate this paper describes actually translates into measurable learning gains for students.

References

Anderson JR, Boyle CF, Reiser BJ (1985) Intelligent tutoring systems. In: Science, pp 456–462

Anthropic (2024) Introducing the Model Context Protocol. https://www.anthropic.com/news/model-context-protocol

Austin J, Johnson DD, Ho J, et al (2021) Structured denoising diffusion models in discrete state-spaces. In: NeurIPS

Bloom BS (1984) The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. Educational Researcher 13(6):4–16

Borchers C, Shou T (2025) Can large language models match tutoring system adaptivity? A benchmarking study. In: Artificial Intelligence in Education (AIED)

Christensen J, Marks J, Shieber S (1995) An empirical study of algorithms for point-feature label placement. ACM Transactions on Graphics 14(3):203–232

Ellson J, Gansner ER, Koutsofios E, et al (2004) Graphviz and Dynagraph – static and dynamic graph drawing tools. In: Graph Drawing Software. Springer, p 127–148

García-Méndez S, de Arriba-Pérez F, Somoza-López MdC (2024) A review on the use of large language models as virtual tutors. Science & Education

Graesser AC, Lu S, Jackson GT, et al (2004) AutoTutor: A tutor with dialogue in natural language. Behavior Research Methods, Instruments, & Computers 36(2):180–192

Guan T, Liu F, Wu X, et al (2024) HallusionBench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models.

In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)

Gupta K, Lazarow J, Achille A, et al (2021) LayoutTransformer: Layout generation and completion with self-attention. In: ICCV

Hu EJ, Shen Y, Wallis P, et al (2022) LoRA: Low-rank adaptation of large language models. In: International Conference on Learning Representations (ICLR)

Hunter JD (2007) Matplotlib: A 2D graphics environment. Computing in Science & Engineering 9(3):90–95

Inoue N, Kikuchi K, Simo-Serra E, et al (2023) LayoutDM: Discrete diffusion model for controllable layout generation. In: CVPR

Jain A, Xie A, Abbeel P (2023) VectorFusion: Text-to-SVG by abstracting pixel-based diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 1911–1920

Jiang Z, Guo J, Sun S, et al (2023) LayoutFormer++: Conditional graphic layout generation via constraint serialization and decoding space restriction. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)

Kasneci E, Sessler K, Küchemann S, et al (2023) ChatGPT for good? on opportunities and challenges of large language models for education. Learning and Individual Differences 103:102274

Khan Academy (2024) Khanmigo: AI-powered tutoring at scale via large language models. Khan Academy Technical Report

Koedinger KR, Anderson JR, Hadley WH, et al (1997) Intelligent tutoring goes to school in the big city. International Journal of Artificial Intelligence in Education 8:30–43

Madaan A, Tandon N, Gupta P, et al (2023) Self-Refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems (NeurIPS)

Mangrulkar S, Gugger S, Debut L, et al (2022) PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft

Markov T, Zhang C, Agarwal S, et al (2023) A holistic approach to undesired content detection in the real world. In: Proceedings of the AAAI Conference on Artificial Intelligence

Mayer RE (2009) Multimedia Learning, 2nd edn. Cambridge University Press

OpenAI (2024a) GPT-4o system card. arXiv preprint arXiv:2410.21276

OpenAI (2024b) Introducing structured outputs in the API. https://openai.com/index/introducing-structured-outputs-in-the-api/

Perez E, Strub F, de Vries H, et al (2018) FiLM: Visual reasoning with a general conditioning layer. In: AAAI

Rafailov R, Sharma A, Mitchell E, et al (2023) Direct preference optimization: Your language model is secretly a reward model. In: Advances in Neural Information Processing Systems (NeurIPS)

Renkl A (2014) The worked examples principle in multimedia learning. In: Mayer RE (ed) The Cambridge Handbook of Multimedia Learning, 2nd edn. Cambridge University Press, p 391–412

Rhasspy contributors (2023) Piper: A fast, local neural text-to-speech system. https://github.com/rhasspy/piper

Rodriguez JA, Puri A, Agarwal S, et al (2025a) StarVector: Generating scalable vector graphics code from images and text. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)

Rodriguez JA, Zhang H, Puri A, et al (2025b) Rendering-aware reinforcement learning for vector graphics generation. arXiv preprint arXiv:250520793

Schick T, Dwivedi-Yu J, Dessi R, et al (2023) Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems (NeurIPS)

Shen J, Pang R, Weiss RJ, et al (2018) Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions (Tacotron 2). In: IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)

Sugiyama K, Tagawa S, Toda M (1981) Methods for visual understanding of hierarchical system structures. IEEE Transactions on Systems, Man, and Cybernetics 11(2):109–125

Sweller J (1988) Cognitive load during problem solving: Effects on learning. Cognitive Science 12(2):257–285

VanLehn K (2006) The behavior of tutoring systems. International Journal of Artificial Intelligence in Education 16(3):227–265

VanLehn K (2011) The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. Educational Psychologist 46(4):197–221

W3C Web Applications Working Group (2024) Web speech API. W3C Working Draft

von Werra L, Belkada Y, Tunstall L, et al (2020) TRL: Transformer reinforcement learning. https://github.com/huggingface/trl

Wolfram S (2009) Wolfram—Alpha: A new computational knowledge engine. https://www.wolframalpha.com/

Wu R, Su W, Ma K, et al (2023) IconShop: Text-guided vector icon synthesis with autoregressive transformers. ACM Transactions on Graphics 42(6)

Wu R, Su W, Liao J (2025) Chat2SVG: Vector graphics generation with large language models and image diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)

Xing X, Zhou H, Wang C, et al (2024) SVGDreamer: Text-guided SVG generation with diffusion model. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 4546–4555

Xing X, Hu J, Liang G, et al (2025) Empowering LLMs to understand and generate complex vector graphics. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp 19487–19497

Xing X, Xue Z, Guan Y, et al (2026) Reason-SVG: Enhancing structured reasoning for vector graphics generation with reinforcement learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)

Yang A, et al (2024) Qwen2.5 technical report. arXiv preprint arXiv:241215115

Yang Y, Cheng W, Chen S, et al (2025) OmniSVG: A unified scalable vector graphics generation model. In: Advances in Neural Information Processing Systems (NeurIPS)

Ye K, Ni W, Krieger M, et al (2020) Penrose: From mathematical notation to beautiful diagrams. In: ACM Transactions on Graphics (SIGGRAPH)

Zala A, Lin H, Cho J, et al (2024) DiagrammerGPT: Generating open-domain, open-platform diagrams via LLM planning. In: Conference on Language Modeling (COLM)

Zhang J, Guo J, Sun S, et al (2023) LayoutDiffusion: Improving graphic layout generation by discrete diffusion probabilistic models. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)

Zheng L, Chiang WL, Sheng Y, et al (2023) Judging LLM-as-a-Judge with MT-Bench and chatbot arena. In: Advances in Neural Information Processing Systems (NeurIPS)